

P064 | Lab C：急件應提前多久送出？

問題：急件應提前多久送出，才不會為省能量而誤期？

急件在佇列

期限倒數
等待越久，越可能誤期

服務窗口 | gate

窗口開啟 → 可以送出
距限接近 → 優先處理
窗口關閉 → 不得送出

本頁讀法 | 先建立問題；下一頁開啟實際檔案，確認哪一行控制急件時間。

queue age | 等待時間

來源 *runner trace*；作用 顯示等待成本；單位 *s*；數字越高代表排隊越久。*deadline* 先判期限，*endpoint J* 只在服務結果後解釋。

P065 | 打開實際檔案：急件門檻 20 秒

先讀原始決策：它只看急件距限與佇列，不改任何值。

student_policy.py | *lab-c-batch-urgent* (只讀)

```
# === LORA EDITABLE: lab-c-batch-urgent ===
BATCH_SIZE = 3      # 佇列達 3 才批次
URGENT_MARGIN_S = 20 # 距限 ≤ 20 秒才急送
# === LORA END EDITABLE: lab-c-batch-urgent ===
# 有急件且距限 ≤ 門檻 → 優先急件
if observation.urgent_pending and
observation.urgent_due_in_s <=
URGENT_MARGIN_S:
    return SEND_URGENT
```

完整動作順序

- 1 窗口關閉 → SLEEP
- 2 急件距限內 → SEND_URGENT
- 3 間隔未到 → REST_DURING_GAP
- 4 品質未穩定 → HOLD
- 5 佇列達 3 → FLUSH_BATCH
- 6 其餘 → WAIT

機制 | 先判窗口與急件；只有前面都未命中，佇列達 3 才進入批次。

URGENT_MARGIN_S | 急件距限門檻

來源 /home/u24/lora-energy-lab/student_policy.py 的 *lab-c-batch-urgent*；單位 s；20 表示距限 20 秒內才走急件分支，較小值較晚、較大值較早。*BATCH_SIZE* = 3、Lab A / B frozen blocks、runner、scenario、schemas、其他常數／函式／檔案均不變。

P066 | 第一輪執行：基準 20 → 候選 5

順序固定：baseline 20 → candidate 5；每次都保存同一對結果檔。

基準組 | baseline 20

先讀本次基準結果，再保留它的配對檔。
bash course.sh run --lab C --case baseline
Windows：course.cmd run --lab C --case baseline

候選組 | candidate 5

先讀本次候選結果，再保留它的配對檔。
bash course.sh run --lab C --case candidate
Windows：course.cmd run --lab C --case candidate

成功訊號 | stdout 印出 `result\_path`；同一 run 目錄保存 `result.json` + `endpoint-replay.json`，先完成 20 再完成 5。

result_path | 結果路徑

來源 runner stdout；值型態 file path；作用 定位本次 result.json 與配對 endpoint-replay.json；判讀路徑相同才算一對，數字以本次收據為準。

P067 | 第一次修改：急件門檻 20 → 5

20 → 5：較小門檻會讓急件分支更晚出現。

修改前 | baseline

`BATCH_SIZE = 3`
`URGENT_MARGIN_S = 20`
距限 ≤ 20 秒 → 急件先送

修改後 | candidate

`BATCH_SIZE = 3`
`URGENT_MARGIN_S = 5`
距限 ≤ 5 秒 → 急件先送

不變 | A/B

file : student_policy.py
block : lab-c-batch-urgent
`BATCH_SIZE = 3` ; A/B
frozen
runner / scenario / schemas
其餘常數、函式、檔案

機制 | 距限是現在到期限的秒數；門檻越小，必須更接近期限才回傳
`SEND\_URGENT`，可能壓低 delivery / deadline。

先讀，再執行 | candidate

先讀：只觀察這次 candidate，不改其他值。命令：bash course.sh run --lab C --case candidate ; Windows 使用 course.cmd 等價命令。成功訊號：stdout 印 result_path，同一目錄有 result.json + endpoint-replay.json；再讀 urgent send、delivery、service / deadline、endpoint J。

P068 | 執行前：把 5 秒預測寫成可反駁句

先寫下可被資料推翻的預測：5 秒可能省電，也可能誤期。

距限 | due

urgent_due_in_s
單位 s；越小
越急

佇列 | queue

queue_size
單位 count；越
高壓力越大

服務／期限

service_pass /
deadline_pass
FAIL 先於 J

端點能量 | J

endpoint_energy_
j
單位 J；低只
代表耗能較少

預測句 | 較晚急件介入可能減少早期 wake／TX；delivery 或 deadline FAIL 即推翻純省能量說法。

證據順序

來源 observation / result summary / endpoint ledger；先讀服務／期限，再回到 packet 與 state 解釋 J。 *urgent_due_in_s* 越小越急； *endpoint_energy_j* 越低只代表耗能較少，不能覆蓋 FAIL。

P069 | 第一次比較：先看服務，再看端點效率

先判服務是否完成，再看端點用了多少能量。

基準組 | margin 20

服務／期限：PASS ／ PASS
送達資料：10,400 bit （越多越完整）
端點能量：10.66 J （J 越低越省）

修改組 | margin 5

服務／期限：FAIL ／ FAIL
送達資料：9,600 bit （少 800 bit ）
端點能量：9.74 J ；效率 bit/J 較高
但 FAIL 先判，不能稱成功

因果解讀 | 少交付 800 bit 且兩個 gate FAIL ；較高 bit/J 不改寫為成功節能。

欄位讀法 | 來源與單位

service_pass ／ *deadline_pass* ：result.json 的布林 gate ，PASS 才往下讀；*delivered_bits* ：result.json 的 bit ，越多越完整；*endpoint_energy_j* ：replay ／ ledger 的 J ，越低越省；*bit_per_j* ：bit/J ，越高效，但不能覆蓋 FAIL 。

P070 | 第二次修改：急件門檻 5 → 30

5 → 30：較大門檻會讓急件分支更早出現。

修改前 | candidate

`URGENT_MARGIN_S = 5`
距限 ≤ 5 秒 → 急件先送
delivery / deadline : FAIL

修改後 | revision

`URGENT_MARGIN_S = 30`
距限 ≤ 30 秒 → 急件先送
較早介入，觀察是否恢復服
務

不變 | A/B

file : student_policy.py
block : lab-c-batch-urgent
`BATCH_SIZE = 3` ; A/B
frozen
runner / scenario / schemas
其餘常數、函式、檔案

機制 | 距限 ≤ 門檻才回傳 `SEND_URGENT`；門檻越大，越早介入，可能增加 wake
/ TX 但保住期限。只允許這一次修訂。

先讀，再執行 | revision

先讀：只觀察這次 revision，不改其他值。命令：`bash course.sh run --lab C --case revision --freeze`；
Windows 使用 `course.cmd` 等價命令。成功訊號：stdout 印 `result_path`，同一目錄有 `result.json` +
`endpoint-replay.json`；再讀 `delivery`、`deadline`、`state ledger`、`endpoint J` 與 `freeze receipt`。

P071 | 執行前：把 30 秒預測寫成取捨句

先比兩種介入時機，再看服務與能量取捨。

門檻 5 | 較晚介入

批次 → 期限 → 急件
較少早期 wake / TX，但 delivery
可能受壓

門檻 30 | 較早介入

急件 → wake / TX → delivery
較可能保住期限，也可能增加 energy
cost

條件式預測 | 只有 delivery、deadline、state ledger 與 endpoint J 同時對上，取捨敘述才成立。

證據欄位 | 來源與高低

delivery / deadline：result 的布林結果，PASS 才算完成；retry / expiry：packet events 的 count，越高表示重試／過期越多；endpoint J：ledger 的 J，越低耗能較少但不能覆蓋 FAIL。

P072 | 依序執行：30 freeze → surprise

順序固定：revision 30 --freeze → surprise；程式內容不再改。

修訂組 | revision 30 freeze

先讀固定後的結果，再保存收據。
bash course.sh run --lab C --case revision
--freeze
stdout → result_path + freeze receipt

保留組 | surprise withheld

先讀新情境的結果；程式與門檻不再改。
bash course.sh run --lab C --case surprise
policy bytes unchanged；讀 withheld
evidence

固定 → 保留 | 新的 case 不新增修改；failure 只縮小 claim，不重新調參。

freeze | 凍結收據

來源 runner receipt；作用 綁定 policy bytes / case；判讀 identity 不符即停止。成功訊號是 stdout
result_path 與 freeze receipt；每一目錄的 result.json、endpoint-replay.json 必須成對，surprise 只讀
matching evidence。

P073 | 三筆結果：基準 20 / 候選 5 / 修訂 30

先把三筆結果排在一起：20、5、30；先看服務，再看端點能量。



因果句 | candidate 少交付且 gate FAIL；revision 在固定條件下恢復 gate，不代表 30 永遠最佳。

比較欄位 | 來源與判讀

service / deadline：result gate，PASS 才往下讀；delivered_bits：bit，越高越完整；endpoint J：J，越低耗能較少；bit/J：效率，越高較好但不能覆蓋 FAIL。數值以 P066 / P072 stdout result_path 為準。

P074 | 保留情境：沿用 30，不重調

另一個情境沿用原程式；低端點能量不代表服務成功。

凍結修訂 | revision

$URGENT_MARGIN_S = 30$
程式內容不變，沿用原條件

保留情境 | same policy

服務／期限：FAIL／FAIL
2,400 bit；5.41 J；3 expired packets

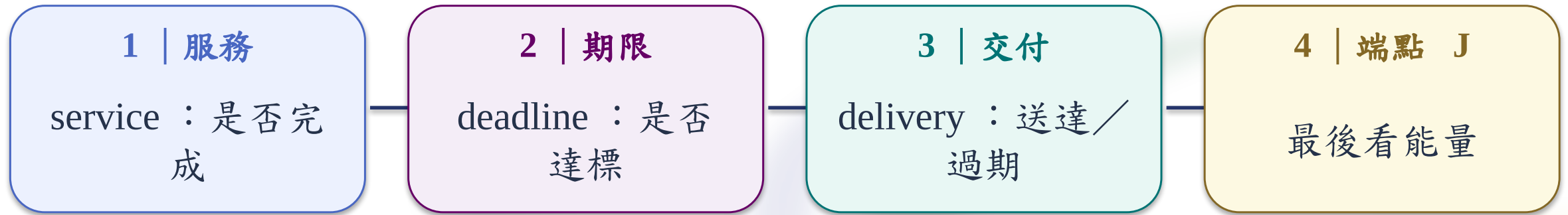
BOUNDARY READING | surprise 低 J 仍是 failure；保存為 frozen policy 的適用邊界。

來源範圍 | 數值判讀

來源 `result.json + endpoint-replay.json`；`delivered_bits` 單位 bit，越高越完整；`endpoint J` 單位 J，越低耗能較少；`service / deadline FAIL` 先判，低 J 不等於成功。

P075 | Lab C 結語：服務／期限先於端點 J

四道門依序檢查；最後才描述端點能量。



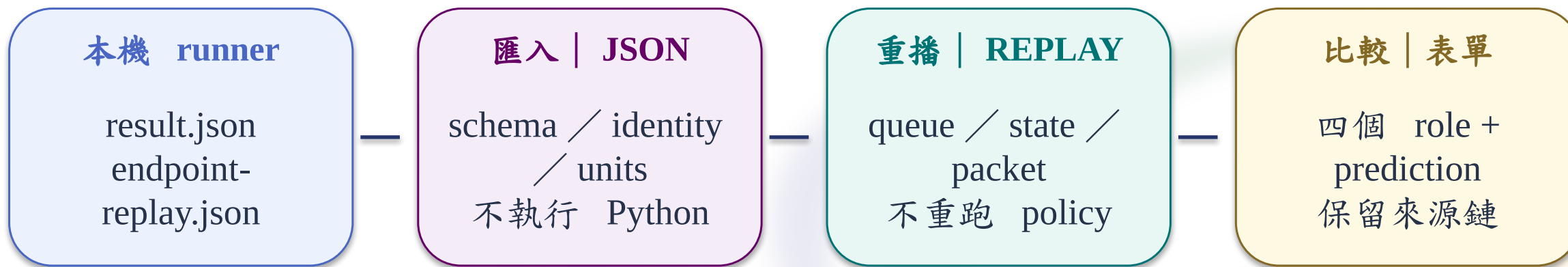
結論 | margin 5 低 J 但 gate FAIL ; margin 30 恢復固定條件 gate ; surprise 暴露窗口限制。

主張邊界

SIMULATED TEACHING DATA ; NOT LIVE ; NOT MEASURED ; NOT CANONICAL-PARITY-VERIFIED 。低 J 不覆蓋服務 FAIL 。

P076 | 網站責任：驗證、重播、保存比較

本機先產生資料；網站只驗證、重播、保存比較。

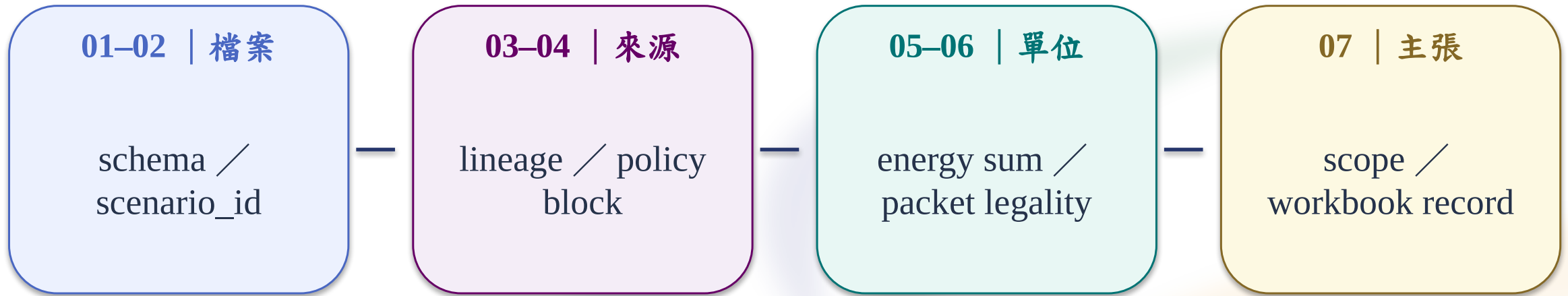


責任邊界 | 網站只接受 JSON artifact；Python policy 與 endpoint result 留在本機 runner。

網站讀法 | 現有 browser record
先讀 service / deadline，再讀 packet delivery / expiry，再讀 Radio / state event，最後讀 endpoint J
與 bit/J；browser accepted 不等同 live measurement，fresh runner parity 待取得。

P077 | 匯入檢查：來源與單位逐層驗證

匯入檔案先逐層驗證；任何失敗都保留原狀。



FAIL CLOSED | 任一層失敗：session / workbook 不變；保存錯誤與 path，回 matching artifact。

scenario_id | 情境識別碼
來源 scenario package；值型態 identifier；作用 綁定同一情境；相同才可比較，缺失或不符即停止。
七層驗證完整後才 materialize replay。

P078 | 事件重播：逐格讀取佇列、動作、狀態、封包

畫面每一格都對應同一條事件記錄；一起讀動作與封包結果。

端點重播 · 選取畫面 1/56 · 0s

無線電 SLEEP	動作 -	佇列數 1	累積端點能量 J 0J
經過時間 0s	接觸識別碼 -	接觸狀態 關閉	連線品質 0

事件讀法

Queue / Action
佇列 → policy 動作

Radio / state
狀態：SLEEP / TX / RX

Packet / service
送達 / 過期 → 期限 / J

重播邊界 | 讀取已驗證 event ledger，不重新執行 policy，也不在瀏覽器推導新數值。

欄位讀法 | 來源、單位、高低

Queue：frame，count，越高待處理越多；Action：policy token，對回程式分支；Radio / state：frame event，確認 SLEEP / TX / RX；endpoint J：ledger，J，越低耗能較少，但服務 FAIL 仍先保留。

P079 | 比較紀錄：四個角色共用來源鏈

比較紀錄把四個角色放在同一條來源鏈，方便重開核對。

實驗/案例	顯示	角色	服務	端點能量 J	來源
A/baseline	目前	基準	FAIL	6.92 J	同情境備用資料

current browser record | execution ledger

四個角色

基準組
baseline 結果

候選組
candidate 修改

修訂組
revision gate

保留組
surprise 邊界

比較紀錄 | 保存 prediction 、 changed line 、 result_path 、 replay 、 gate 、 endpoint scope 與 recovery note 。

provenance | 來源鏈

來源 /course workbook ；值型態 saved record ；作用 保存四個角色的 lineage ；缺必要 evidence 就維持 INCOMPLETE ，不補寫答案。

P080 | 保存與重開：先核對身份，再恢復來源鏈

保存、關閉、重開都先核對身份，再恢復同一條來源鏈。

01 | 保存 SAVE

保存預測＋結果／重播

02 | 關閉 CLOSE

不改 runner 檔案

03 | 重開

OPEN ／ REBIND：核對情境＋端點

完成度 | 身份、prediction、result／replay 齊全才 COMPLETE；缺件維持 INCOMPLETE 並 recovery，不改寫 FAIL。

COMPLETE／INCOMPLETE | 完成度狀態

來源 workbook validator；值型態 status enum；作用 反映必要 evidence 是否齊全；判讀 COMPLETE 不等於 service PASS，也不等於 scientific parity。